MalAI: AI-Powered Decompilation Plugin for Ghidra

Luqman Al Hasni, Long Nguyen

University of CINCINNATI

Background

•**Reverse engineers** currently rely on decompilers in tools like Ghidra to generate C-Code from ASM for Malware Analysis
•Ghidra's current decompilation process is semantically accurate but lacks readability and can be difficult to understand
• Recent research papers have shown promise in LLMs ability to generate more readable C-Code for decompilation
• MalAI explores whether fine-tuned LLMs can assist decompilation for malware analysis and reverse engineering workflows

Aims

- Integrate a **fine-tuned LLM** to decompile Ghidra's disassembled code to C to improve the following:
  1. **Readability**:
     - **Ghidra's** decompiled C code is often semantically correct but could improve in readability and interpretability
  2. **Complexity**:
     - **DeepSeek Coder's** decompilation output can be more readable but struggles with complex inputs

Challenges

- **LLM Development Familiarity:** Navigating technical constraints of LLM finetuning and LLM integrations.
- **Research-Driven development**: LLM4Decompile is an ongoing research; consequently, new approaches were proposed by the team in the past two semesters.
- **Model Complexity and Readability:** The model needed to exceed in both complexity and readability in comparison to Deepseek-Coder and Ghidra respectively.
- **Ghidra Integration:** Lack of familiarity with Ghidra development environment and plugin development introduced new constraints

Methods

Datasets & Approach

- LLM Decompilation:
  - **Approach:**
    - ASM – C pairs are utilized to conduct the the Deepseek-Coder LLM finetuning process.
    - The LLM is given a x64 ASM as input with no additional prompts and the model is expected to output the correct decompiled C code for the given ASM.
  - **Used Datasets:**
    - Decompile Bench
    - Exebench
- Malware Detection:
  - **Used Datasets:**
    - Dike Dataset

Methods

Decompilation & Malware Detection Models Training

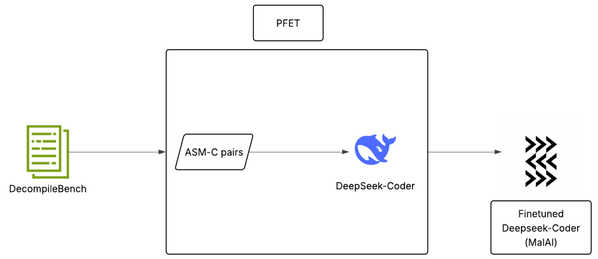

Figure 1: Deepseek-Coder Finetuning Diagram

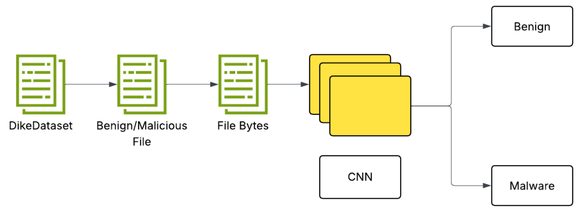

Figure 2: Malware Detection Training Diagram

Plugin Architecture

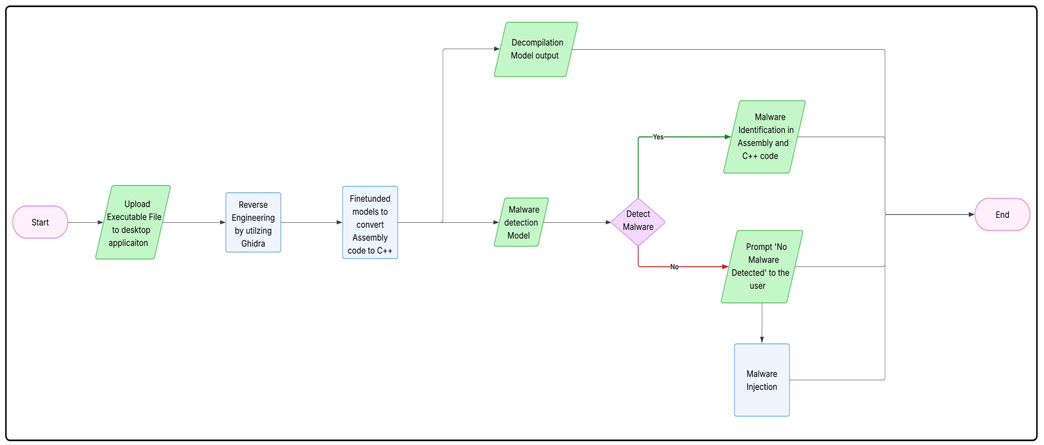

Figure 3: MalAI Plugin Flowchart

References

[1] Tan, H., Luo, Q., Li, J., and Zhang, Y. **LLM4Decompile: Decompiling Binary Code with Large Language Models.** *arXiv preprint* arXiv:2403.05286, 2024.
[2] Tan, H., Li, W., Tian, X., Wang, S., Liu, J., Li, J., and Zhang, Y. **SK2Decompile: LLM-based Two-Phase Binary Decompilation from Skeleton to Skin.** *arXiv preprint* arXiv:2509.22114, 2025.
[3] Armengol-Estapé, J., Woodruff, J., Dos Reis, G., Black, S., O'Boyle, M. F. P., and Cummins, C. **ExeBench: An ML-Scale Dataset of Executable C Functions.** In *Proceedings of the 19th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes (MPLR)*, pp. 50–59, 2022.
[4] Tan, H., Tian, X., Qi, H., Liu, J., Gao, Z., Wang, S., Luo, Q., Li, J., and Zhang, Y. **Decompile-Bench: Million-Scale Binary-Source Function Pairs for Real-World Binary Decompilation.** *arXiv preprint* arXiv:2505.12668, 2025.

Results

Results: Assembly Comparisons

Ghidra / Baseline

Example Decompiled Output

```
uStack_c = 0;
 for (uStack_10 = 0; uStack_10 < param_2; uStack_10 = uStack_10 + 1) {
   if (((uint)(param_1 + (longlong)uStack_10 * 4) & 1) == 0) {
     uStack_c = uStack_c + (int)(param_1 + (longlong)uStack_10 * 4);
   }
 }
 return uStack_c;
```

Typical Characteristics:

- Generic variable names reduce readability
- Function intent is less immediately clear
- Includes low-level type/casting artifacts
- Requires more analyst interpretation

MalAI (Fine-Tuned Model)

Example Decompiled Output

```
int sum_even_elements(int* arr, int n) {
   int i = 0;
   int sum = 0;
   while (i < n) {
      if ((arr[i] & 0x1) == 0) {
         sum += arr[i]; }
      i++;
   }
   return sum; }
```

Typical Characteristics:

- More descriptive names improve readability
- High-level function purpose is clearer
- Cleaner C-style structure with less clutter
- Faster for analysts to interpret at a glance

Figure 4: Assembly Generation Comparison between Ghidra and MalAI Decompilation Model

Results: Interface

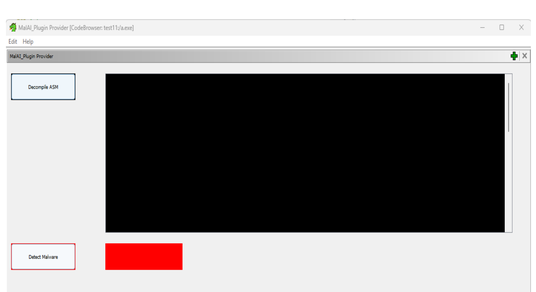

Figure 5 : MalAI Plug-in Interface Before Pressing The Decompilation Button

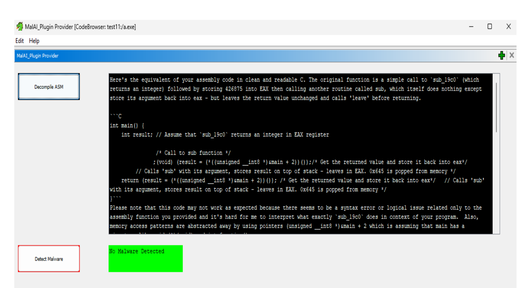

Figure 6: MalAI Plug-in Interface After Pressing The Decompilation Button

Conclusions

- MalAI demonstrates that a fine-tuned LLM can be integrated into Ghidra to support decompilation and reverse engineering workflows
- Preliminary results show that LLM-assisted decompilation can improve the readability of generated C-like code on selected examples
- The project establishes a working foundation for studying semantic correctness, readability, and practical usefulness in AI-assisted decompilation

Next Steps

- Expand ASM-to-C training data for broader function coverage
- Implement semantic evaluation using execution-based testing
- Benchmark against Ghidra and base DeepSeek-Coder outputs
- Improve consistency on complex reverse-engineering examples
- Refine Ghidra plugin usability for analyst workflows